

Lab1 — Denoising Diffusion Probabilistic Models

學號：114550109 姓名：詹佳翰

項目	結果
Task 1 Chamfer Distance (門檻 < 20)	6.7868
Task 2 最佳 FID (quad schedule + noise predictor)	0.8252

Task 1 — Swiss Roll (2D DDPM)

1.1 TODO 實作與說明

TODO #1 SimpleNet (network.py)

```
dims = [dim_in] + list(dim_hids) + [dim_out] # [2, 128, 128, 128, 2]
self.layers = nn.ModuleList([TimeLinear(dims[i], dims[i+1], num_timesteps)
                              for i in range(len(dims) - 1)])

self.act = nn.SiLU()
# forward
for i, layer in enumerate(self.layers):
    x = layer(x, t)
    if i < len(self.layers) - 1: # 輸出層不加 activation
        x = self.act(x)
```

- 網路是 4 層 TimeLinear (2→128→128→128→2)，輸入 x_t ，輸出同維度的雜訊預測 $\epsilon_\theta(x_t, t)$ 。
- TimeLinear 會把 timestep t 做 embedding 後乘在線性層的輸出上，讓每一層都知道目前的雜訊強度。因為每層都要額外吃 t ，所以用 nn.ModuleList 自己跑迴圈，不能用 nn.Sequential。
- 中間層接 SiLU。最後一層不加 activation，因為要預測的 $\epsilon \sim \mathcal{N}(0, I)$ 可正可負、沒有值域限制。

TODO #2 q_sample (forward process)

由 $q(x_t|x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$ 用 reparameterization 一步取樣：

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

```
alphas_prod_t = extract(self.var_scheduler.alphas_cumprod, t, x0) # (B,1)
xt = alphas_prod_t.sqrt() * x0 + (1 - alphas_prod_t).sqrt() * noise
```

extract 會依每筆資料各自的 t 取出 $\bar{\alpha}_t$ ，並 reshape 成 (B,1)，和 (B,2) 的 x_0 broadcasting 相乘。因為有這條閉式解，訓練時不需要真的迭代 t 步。

TODO #3 p_sample (reverse 一步 $x_t \rightarrow x_{t-1}$)

DDPM Eq. 11：

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right), \quad \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t, \quad x_{t-1} = \mu_\theta + \sqrt{\tilde{\beta}_t} z$$

```

eps_theta = self.network(xt, t) # 1. 預測雜訊
mean = (xt - eps_factor * eps_theta) / alpha_t.sqrt() # 2. posterior mean
var = (1 - alpha_bar_t_prev) / (1 - alpha_bar_t) * beta_t # 3. posterior variance
z = torch.randn_like(xt)
nonzero_mask = (t != 0).float().view(-1, *(([1] * (xt.dim() - 1))))
x_t_prev = mean + nonzero_mask * var.sqrt() * z # 4. t=0 不加雜訊

```

- $\text{eps_factor} = \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} = \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}$ ，就是 Eq. 11 裡 ϵ_θ 前的係數。
- 變異數用 posterior variance $\tilde{\beta}_t$ (論文指出 $\sigma_t^2 = \beta_t$ 或 $\tilde{\beta}_t$ 結果相近)。
- nonzero_mask：最後一步 $t = 0$ 直接輸出 mean，不再加隨機雜訊，得到乾淨的 x_0 。

TODO #4 p_sample_loop (DDPM Algorithm 2)

```

xt = torch.randn(shape).to(self.device) # x_T ~ N(0, I)
for t in range(self.var_scheduler.num_train_timesteps - 1, -1, -1): # 999 → 0
    xt = self.p_sample(xt, t)
x0_pred = xt

```

從標準常態分佈出發，連續呼叫 p_sample 做 1000 次去噪，最後得到的就是生成樣本。

TODO #5 compute_loss (DDPM Eq. 14, simple loss)

$$L_{\text{simple}} = \mathbb{E}_{t, x_0, \epsilon} \left\| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, t) \right\|^2$$

```

t = torch.randint(0, num_train_timesteps, size=(batch_size,)) # 1. 均勻抽 t
eps = torch.randn_like(x0) # 2. 真正加進去的雜訊
x_t = self.q_sample(x0, t, noise=eps)
eps_pred = self.network(x_t, t) # 3. 網路預測雜訊
loss = F.mse_loss(eps_pred, eps) # 4. MSE

```

每個 batch 對每筆資料獨立抽一個 t ，所以一個 batch 同時訓練到各種雜訊強度。網路的目標是從 x_t 還原出當初加進去的 ϵ 。

1.2 q_sample 視覺化

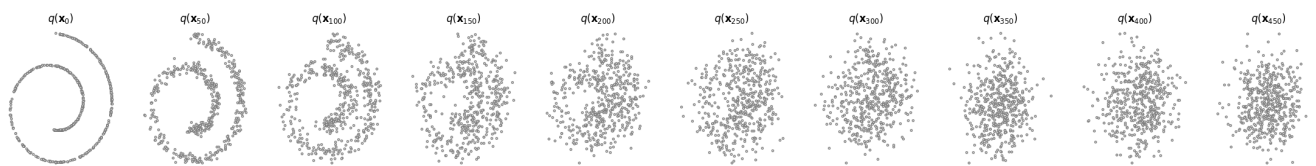


Fig. 1: $q(x_t|x_0)$ ， $t = 0, 50, \dots, 450$ 。螺旋 (Swiss roll) 隨 t 增加逐漸被雜訊蓋過，最後趨近各向同性的高斯分佈，和投影片的參考圖一致。

1.3 Loss 曲線與生成分佈

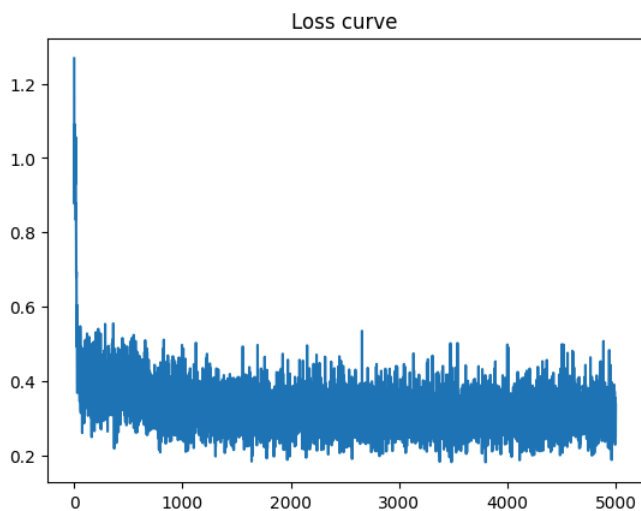


Fig. 2 : 訓練 loss (5000 iterations)。loss 很快從約 1 降下來，之後在低點震盪（最後約 0.34）。

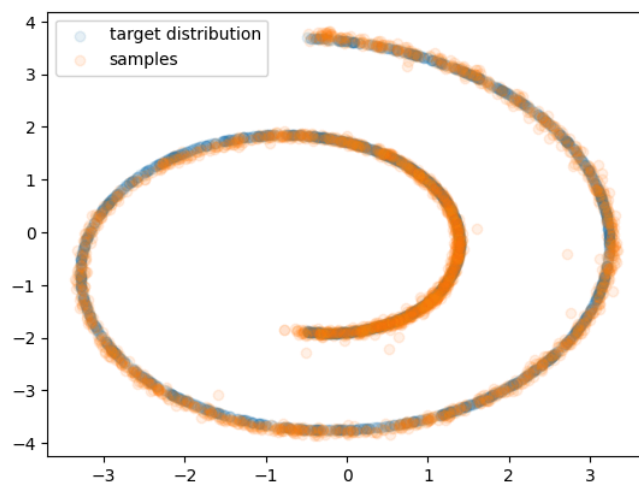


Fig. 3 : 藍色為目標分佈，橘色為 DDPM 生成的 2048 個點。
Chamfer Distance = 6.7868 (< 20)。

生成的點幾乎完整覆蓋整條螺旋，只有少量點落在螺旋外。simple loss 的最小值不是 0：同一個 x_t 可能來自不同的 (x_0, ϵ) 組合，所以 ϵ 本身無法被完全確定，loss 收斂到一個非零的平台是正常的。

Task 2 — Image Generation (AFHQ 64×64)

2.1 TODO 實作與說明

TODO #1 add_noise (scheduler.py)

```
alpha_bar_t = extract(self.alphas_cumprod, t, x_0) # [B,1,1,1]
x_t = alpha_bar_t.sqrt() * x_0 + (1 - alpha_bar_t).sqrt() * eps
```

和 Task 1 的 q_sample 是同一條公式 $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$ ，只是資料變成 $[B,C,H,W]$ 的影像，`extract` 會 reshape 成 $[B,1,1,1]$ 做 broadcasting。函式同時回傳 `eps`，讓 `loss` 可以拿它當 ground truth。

TODO #2 Cosine beta schedule (Nichol & Dhariwal, 2021)

$$f(t) = \cos^2\left(\frac{t/T + s}{1 + s} \cdot \frac{\pi}{2}\right), \quad \bar{\alpha}_t = \frac{f(t)}{f(0)}, \quad \beta_t = \min\left(1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}}, 0.999\right), \quad s = 0.008$$

```
s = 0.008
steps = torch.arange(num_train_timesteps + 1, dtype=torch.float64) # t = 0..T
f = torch.cos(((steps / num_train_timesteps) + s) / (1 + s) * torch.pi / 2) ** 2
alphas_cumprod = f / f[0] # \bar{\alpha}_0 = 1
betas = 1 - alphas_cumprod[1:] / alphas_cumprod[:-1] # T 個 \beta
betas = betas.clamp(max=0.999).float()
```

- cosine schedule 是直接設計 $\bar{\alpha}_t$ ，再反推 β_t 。算 T 個 β_t 需要 $T + 1$ 個 $\bar{\alpha}$ ，所以 `steps` 取 $0..T$ 。
- 除以 $f(0)$ 讓 $\bar{\alpha}_0 = 1$ ；小位移 s 避免 t 接近 0 時 β_t 太小。
- $t \rightarrow T$ 時 $\bar{\alpha}_t \rightarrow 0$ ，比值會讓 $\beta_t \rightarrow 1$ ，所以 clip 到 0.999。用 `float64` 計算，避免 $\bar{\alpha}$ 很小時相除產生數值誤差，最後再轉回 `float32`。

TODO #3 三種 predictor 的 reverse step

三種參數化最後都要算出 posterior $q(x_{t-1}|x_t, x_0) = \mathcal{N}(\tilde{\mu}_t, \tilde{\beta}_t I)$ (DDPM Eq. 7)：

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}x_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}x_t, \quad \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}\beta_t$$

差別只在網路輸出的是哪一個量，所以我把共用的部分抽成 helper：

```

def _alpha_bar_prev(self, t, x_t):
    #  $\bar{\alpha}_{t-1}$ ,  $t = 0$  時定義為 1
    prev = extract(self.alphas_cumprod, (t - 1).clamp(min=0), x_t)
    return torch.where((t == 0).view(prev.shape), torch.ones_like(prev), prev)

def _posterior_mean(self, x_t, t, x0_pred):
    #  $\tilde{\mu}_t$ 
    return (ab_prev.sqrt() * beta_t / (1 - ab_t) * x0_pred
            + alpha_t.sqrt() * (1 - ab_prev) / (1 - ab_t) * x_t)

def _posterior_variance(self, x_t, t):
    #  $\tilde{\beta}_t$ 
    return (1 - ab_prev) / (1 - ab_t) * beta_t

def _add_posterior_noise(self, mean, var, t):
    #  $t = 0$  不加雜訊
    z = torch.randn_like(mean)
    nonzero_mask = (t != 0).float().view(-1, *[1] * (mean.dim() - 1)))
    return mean + nonzero_mask * var.sqrt() * z

def _posterior_sample(self, x_t, t, x0_pred):
    x0_pred = x0_pred.clamp(-1, 1)
    return self._add_posterior_noise(self._posterior_mean(x_t, t, x0_pred),
                                     self._posterior_variance(x_t, t), t)

```

- **noise predictor**：先由 $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$ 反解出 $\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t}\epsilon_\theta}{\sqrt{\bar{\alpha}_t}}$ ，再丟進 `_posterior_sample`。

```

x0_pred = (x_t - (1 - alpha_bar_t).sqrt() * eps_theta) / alpha_bar_t.sqrt()
sample_prev = self._posterior_sample(x_t, t, x0_pred)

```

- **x0 predictor**：網路直接輸出 $\hat{x}_0$ ，直接呼叫 `self._posterior_sample(x_t, t, x0_pred)`。
- **mean predictor**：網路直接輸出 $\mu_\theta \approx \tilde{\mu}_t$ ，只需要補上 $\tilde{\beta}_t$ 的雜訊：
`_add_posterior_noise(mean_theta, _posterior_variance(x_t, t), t)`。
- `clamp(-1,1)`：訓練影像被 normalize 到 $[-1, 1]$ ， $\hat{x}_0$ 超出範圍一定是錯的，強制拉回可以避免誤差在 1000 步裡累積放大。
- $\bar{\alpha}_{-1} = 1$ ：在 $t = 0$ 時 $\tilde{\beta}_0 = 0$ 、 $\tilde{\mu}_0 = \hat{x}_0$ ，最後一步直接輸出預測的乾淨影像。
- `step()` 會先把 t 統一轉成 device 上的 1-D tensor，再依 predictor 字串分派到上面三個函式。

TODO #4 三種 predictor 的 loss (model.py)

predictor	網路輸出	regression target
noise (已提供)	$\hat{\epsilon}_\theta(x_t, t)$	ϵ
x0	$\hat{x}_\theta(x_t, t)$	x_0
mean	$\mu_\theta(x_t, t)$	$\tilde{\mu}_t = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1 - \alpha_t}}\epsilon\right)$

```

def get_loss_x0(self, x0, class_label=None, noise=None):
    t = self.var_scheduler.uniform_sample_t(B, x0.device)
    x_t, _ = self.var_scheduler.add_noise(x0, t, eps=noise)
    x0_pred = self.network(x_t, t)
    return F.mse_loss(x0_pred, x0)

def get_loss_mean(self, x0, class_label=None, noise=None):
    t = self.var_scheduler.uniform_sample_t(B, x0.device)
    x_t, eps = self.var_scheduler.add_noise(x0, t, eps=noise)
    mean_pred = self.network(x_t, t)
    mean_true = (x_t - beta_t / (1 - alpha_bar_t).sqrt() * eps) / alpha_t.sqrt()
    return F.mse_loss(mean_pred, mean_true)

```

mean predictor 的 target 用的是閉式 $\tilde{\mu}_t(x_t, x_0)$ 。把 $x_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon}{\sqrt{\bar{\alpha}_t}}$ 代入 Eq. 7 化簡，就得到上面只含 x_t 與 ϵ 的形式 (DDPM Eq. 10)，和 Eq. 7 數學上完全相等，而且直接用真正加入的 ϵ ，不需要再算一次 x_0 。

正確性驗證：pytest tests/test_todo.py 共 28 passed，數值與助教提供的參考輸出一致。

2.2 實驗設定

參數	值	參數	值
train_num_steps	100,000	batch_size	16
T	1000	β_1, β_T (linear / quad)	1e-4, 0.02
resolution	64x64	warmup_steps	200
sampler	DDPM (1000 steps)	FID 樣本數	500 張 vs. AFHQ val (eval set)

共訓練 5 組模型：{linear, quad, cosine} × noise predictor，以及 linear × {x0, mean} predictor。

2.3 三種 beta schedule 的比較

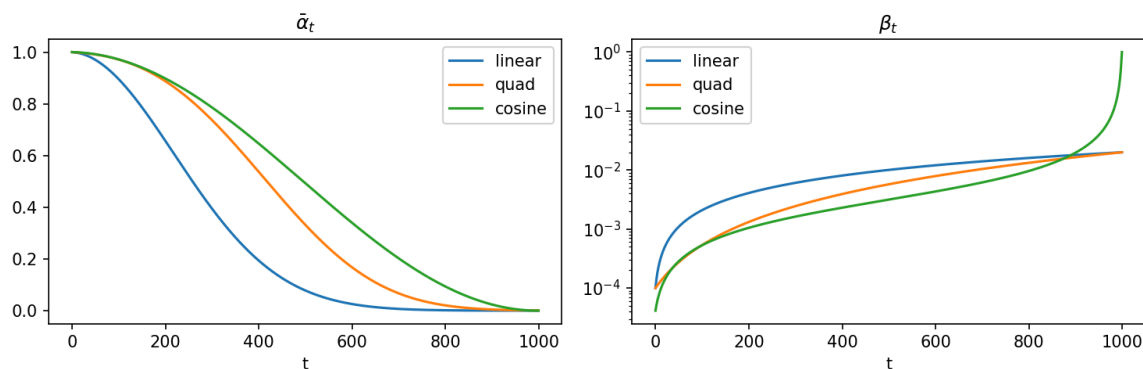


Fig. 4：三種 schedule 的 $\bar{\alpha}_t$ (左) 與 β_t (右, log scale)。

schedule	$\bar{\alpha}_{500}$	$\bar{\alpha}_T$
linear	0.079	4.0e-5
quad	0.333	7.3e-4
cosine	0.494	2.4e-9

Reverse trajectory (最終 checkpoint，左 $t = T \rightarrow$ 右 $t = 0$ ，每格約間隔 111 步)

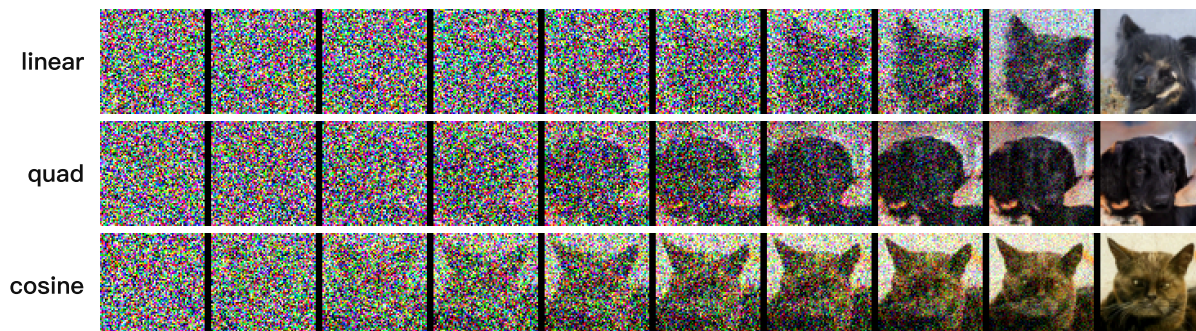


Fig. 5：三種 schedule (皆為 noise predictor) 的去噪軌跡。

每組 8 張生成結果



schedule (noise predictor)	FID ↓
linear	14.3071
quad	0.8252
cosine	6.6765

觀察與討論：

- **linear** : $\bar{\alpha}_t$ 下降得最快， $t = 500$ 時已經只剩 $\bar{\alpha} \approx 0.08$ ，所以 reverse 軌跡的前 6~7 格幾乎都是純雜訊，動物輪廓到最後 3 格才突然浮現。大部分的 timestep 花在「幾乎看不到訊號」的區間，真正決定影像結構和細節的步數比較少。
- **quad** : β_t 從 $\sqrt{\beta}$ 線性內插再平方，前段的 β 比 linear 小很多， $\bar{\alpha}_t$ 衰減較慢 ($\bar{\alpha}_{500} \approx 0.33$)。軌跡約在中段（第 5 格）就出現輪廓，之後逐步補上細節。
- **cosine** : $\bar{\alpha}_t$ 在中段近似線性下降、兩端平緩，雜訊強度在各 timestep 分佈最均勻。軌跡最早（第 3~4 格）出現輪廓，整個過程是由粗到細平滑演變。另外 $\bar{\alpha}_T \approx 2 \times 10^{-9}$ ， x_T 最接近純高斯，和取樣起點 $\mathcal{N}(0, I)$ 最一致。
- 從生成結果看，三種 schedule 都能產生清楚的動物臉。FID 排名為 quad (0.83) < cosine (6.68) < linear (14.31)，三者都低於 15，但 linear 明顯最差。我也在本機用同一個 AFHQ Inception 權重計算各組生成圖之間的 FID：quad 與 cosine 之間只有 3.7，但兩者 and linear 之間分別是 16.5 和 19.2，表示 quad / cosine 的輸出分佈很接近，linear 則偏離較多。原因應該就是上面提到的：linear 把太多步數浪費在高雜訊區，對細節的建模較不足。

訓練過程中的 trajectory (linear + noise)

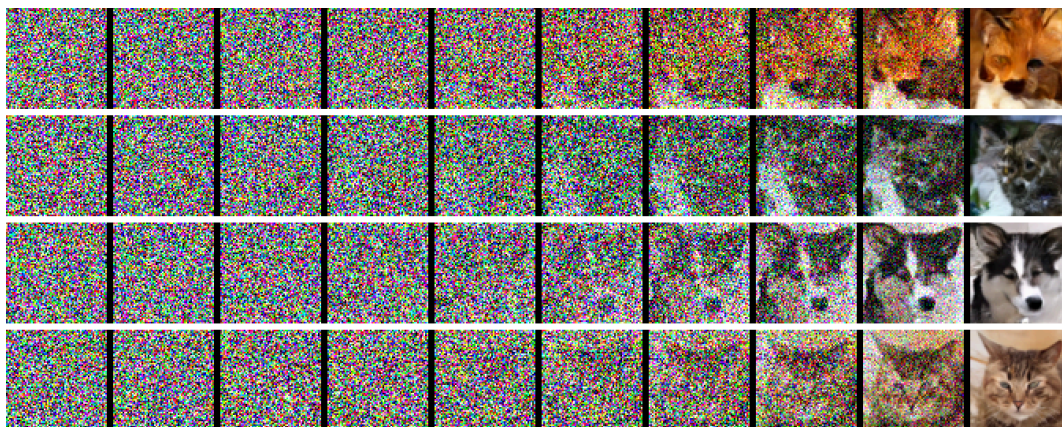


Fig. 6：由上到下為訓練 step 5k、20k、50k、95k 時的去噪軌跡。5k 步已經有模糊的色塊與輪廓，隨訓練進行細節（眼睛、毛色、背景）越來越清楚。

2.4 三種 predictor 的比較 (linear schedule)



Fig. 7：三種 predictor 各 8 張生成結果（共 24 張）。

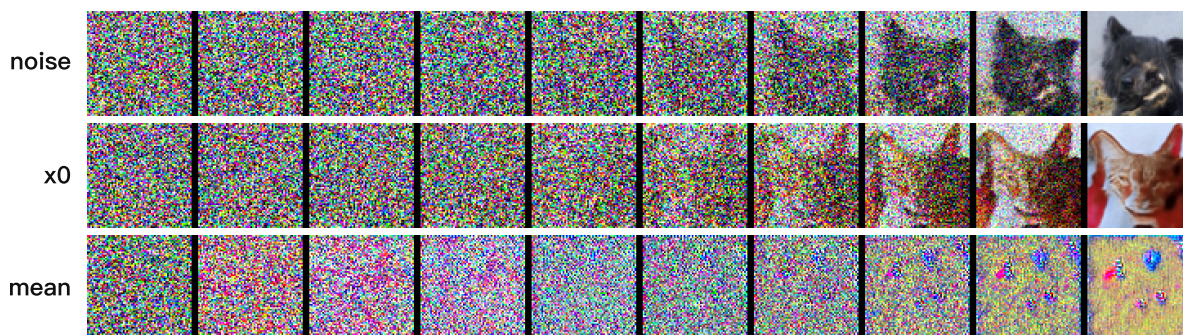


Fig. 8：三種 predictor 的去噪軌跡。

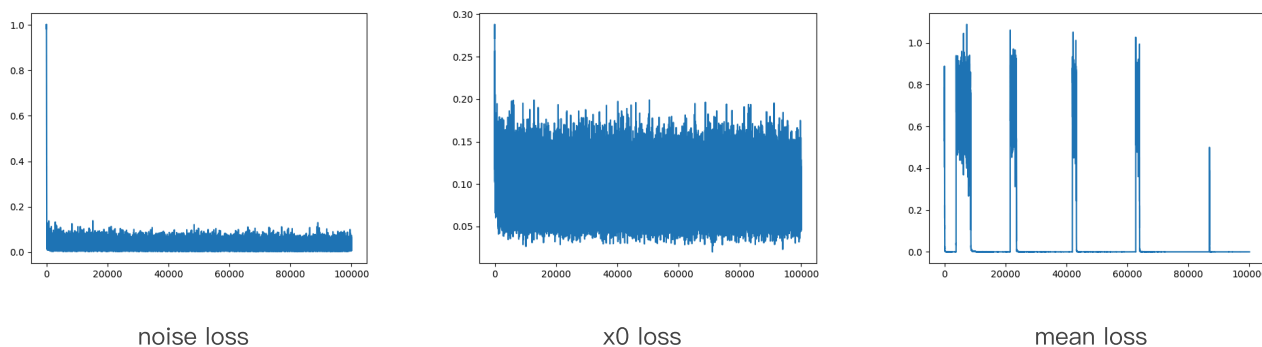


Fig. 9：三種 predictor 的訓練 loss（target 尺度不同，數值不能直接互比）。

predictor (linear schedule)	FID ↓
noise	14.3071
x0	13.8030
mean	49.4326

觀察與討論：

- **noise predictor**：target $\epsilon \sim \mathcal{N}(0, I)$ ，不論 t 為何尺度都固定為單位變異數，對網路來說是一個條件良好的回歸問題。loss 很快降到 0.1 以下並穩定震盪，生成結果清楚、多樣（貓、狗、野生動物都有）。
- **x0 predictor**：同樣能生成可辨識的動物臉，但整體紋理偏平滑、像塗抹過的水彩，毛髮等高頻細節較少，部分樣本的臉部結構也不自然（Fig. 7 x0 列第 3、4、7 張五官扭曲或殘缺）。原因是 t 很大時要從幾乎純雜訊直接猜整張 x_0 ，MSE 的最佳解是「所有可能 x_0 的條件平均」，所以預測偏模糊。loss 維持在 0.05~0.15 之間，比 noise 版本高、震盪也大。不過因為取樣時每一步只取 $\tilde{\mu}_t$ 的一小部分 $\hat{x}_0$ ，再加

上 $\text{clamp}(-1,1)$ ，錯誤能逐步被修正，最終品質仍然不錯。

值得注意的是，x0 的 FID (13.80) 反而比 noise (14.31) 略低。我的解讀是：評估用的 Inception 是在 AFHQ 上 fine-tune 過的（最後接 3 類 cat/dog/wild 分類頭），特徵偏重「物種、臉部整體結構與色彩」這類語意資訊，對毛髮等高頻紋理不敏感。x0 版本的模糊正好落在這個指標不太懲罰的地方，而它的語意分佈和真實資料相當接近（本機計算 x0 與 quad、cosine 生成圖之間的 FID 只有 9.3、4.0，比 noise 與它們之間的 16.5、19.2 還小）。因此在這個 FID 下兩者差距不大，但肉眼看 noise predictor 的細節明顯較好，FID 也不能完全代表影像的清晰度。

- **mean predictor**：完全失敗，輸出是帶色塊的雜訊圖，軌跡也從頭到尾沒有出現影像結構。從 loss 曲線可以看出原因：

1. $\tilde{\mu}_t = \frac{1}{\sqrt{\alpha_t}} x_t - \frac{\beta_t}{\sqrt{\alpha_t} \sqrt{1-\alpha_t}} \epsilon$ ，而 $\beta_t \leq 0.02$ ，所以 target 幾乎就是 x_t 本身，真正有用的去噪資訊只佔很小的尺度。loss 大多數時候接近 0（網路只要學會近似恆等映射就能拿到很低的 loss），梯度對「該往哪個方向去噪」幾乎沒有指導力。
2. loss 曲線多次突然跳到約 1 再掉回來，代表訓練不穩定、會反覆發散。
3. 取樣時每一步的微小誤差不會像 x0 / noise 版本那樣經過 clamp 修正，1000 步累積之後完全偏離資料流形。

這個結果和 DDPM 論文 Table 2 一致：作者指出用 unweighted MSE 直接預測 $\tilde{\mu}$ 會 *training unstable*，只有用真正的 variational bound（含 timestep 權重）才能訓練；這也是論文改用 ϵ 參數化的主要動機。

2.5 最佳 FID 截圖

最佳結果為 quad schedule + noise predictor, FID = 0.8252 (< 15)。

```
[dndn] jzhan1027@sgpu7:/project/jzhan1027/Lab1-DGUV/image_diffusion_todo$ CUDA_VISIBLE_DEVICES=0 python fid/measure_fid.py data/afhq/eval samples/quad_noise
```

```
Calculating FID given paths data/afhq/eval and samples/quad_noise...  
/home/jzhan1027/miniconda3/envs/dgpn/lib/python3.9/site-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.  
  warnings.warn(  
/home/jzhan1027/miniconda3/envs/dgpn/lib/python3.9/site-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing 'weights=None'.  
  warnings.warn(msg)  
/home/jzhan1027/miniconda3/envs/dgpn/lib/python3.9/site-packages/torchvision/models/inception.py:43: FutureWarning: The default weight initialization of inception_v3 will be changed in future releases of torchvision. If you wish to keep the old behavior (which leads to long initialization times due to scipy/scipy#11299), please set init_weights=True.  
  warnings.warn(  
100% ████████████████████████████████████████████████████████████████████████████ | 24/24 [00:01<00:00, 12.44it/s]  
100% ████████████████████████████████████████████████████████████████████████████ | 8/8 [00:00<00:00, 9.21it/s]
```

```
FID: 0.8251655051888704
```

Fig. 10 : quad + noise 的 FID 計算結果 (python fid/measure_fid.py data/afhq/eval samples/quad noise)。

```
(ddpm) jzhan1027@gpu7:/project/jzhan1027/Lab1-DDPM/image_diffusion_todo$ CUDA_VISIBLE_DEVICES=0 python fid/measure_fid.py data/afhq/eval samples/linear_noise
Calculating FID given paths data/afhq/eval and samples/linear_noise...
/home/jzhan1027/miniconda3/envs/ddpm/lib/python3.9/site-packages/torchvision/models/_utils.py:288: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.
  warnings.warn(
/home/jzhan1027/miniconda3/envs/ddpm/lib/python3.9/site-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing 'weights=None'.
  warnings.warn(msg)
/home/jzhan1027/miniconda3/envs/ddpm/lib/python3.9/site-packages/torchvision/models/inception.py:43: FutureWarning: The default weight initialization of inception_v3 will be changed in future releases of torchvision. If you wish to keep the old behavior (which leads to long initialization times due to scipy/scipy#11299), please set init_weights=True.
  warnings.warn(
100%|██████████████████████████████████████████████████████████| 24/24 [00:02<00:00, 11.85tit/s]
100%|██████████████████████████████████████████████████████████| 8/8 [00:01<00:00, 5.85tit/s]
FID: 14.387072276741867
```

Fig. 11 : linear + noise 的 FID 計算結果 (14.3071, 同樣 < 15)。

比對 eval set (AFHQ val 經 dataset.py 轉成 64x64, 約 1500 張, 對應 24 個 batch) 與 500 張生成影像 (8 個 batch)。FID 使用助教提供、在 AFHQ 上 fine-tune 過的 Inception-v3 特徵。